Scales & Coordinate Systems

Workshop 2 — Module 4

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

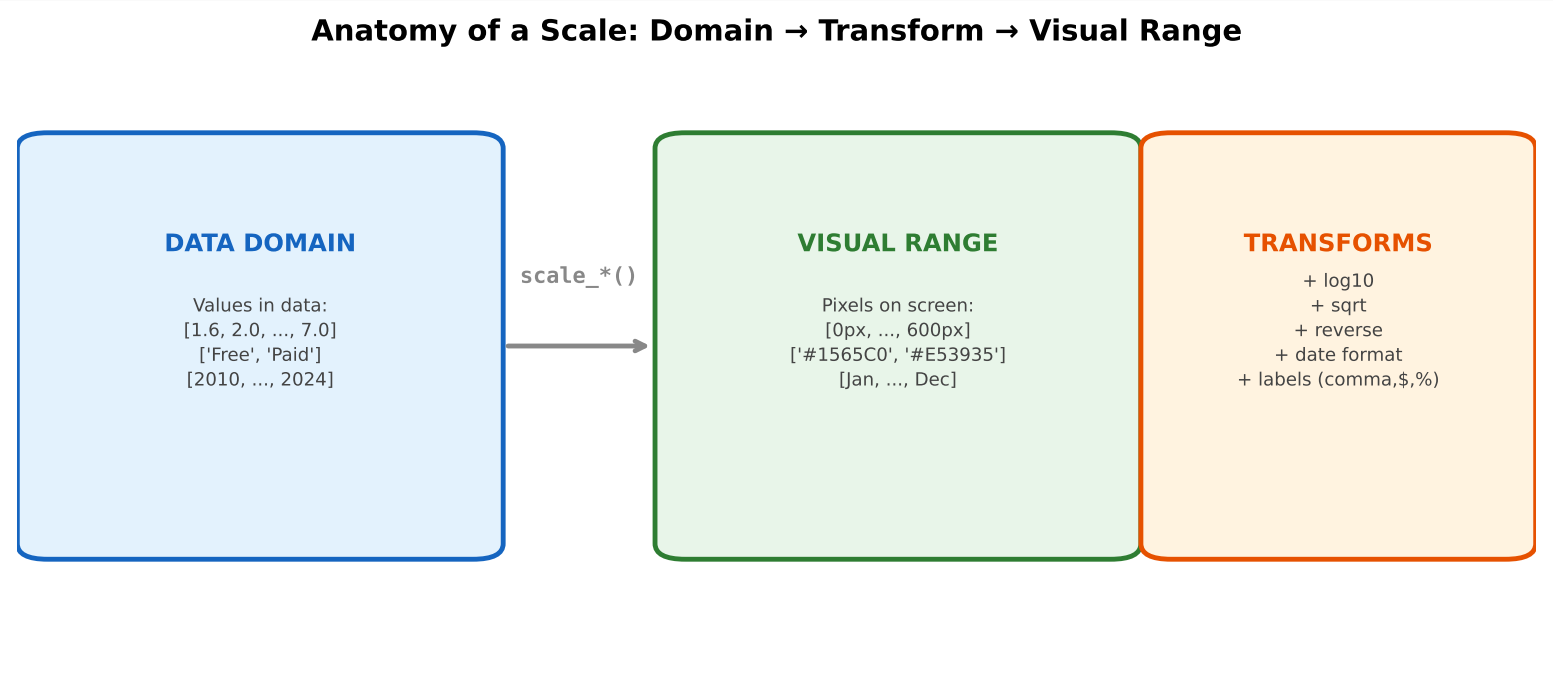
Module Roadmap

1. Anatomy of a scale: domain $\rightarrow$ transform $\rightarrow$ visual range
2. Position scales: continuous, log, discrete, date
3. Colour scales: manual, Brewer, viridis, gradient2
4. Axis label formatting: comma, dollar, percent
5. Coordinate systems: Cartesian, flip, polar, fixed
6. Zooming safely: `coord_cartesian` vs scale `limits`

Learning Outcome
You will control how every data value maps to a visual property — position, colour, size — and how the coordinate system frames the chart.

Anatomy of a Scale

Domain → Transform → Visual Range



Data Insight

A scale answers three questions: (1) What are the input values? (domain), (2) Should they be transformed? (log, sqrt, reverse), (3) What visual output do they produce? (pixel position, hex colour, point size).

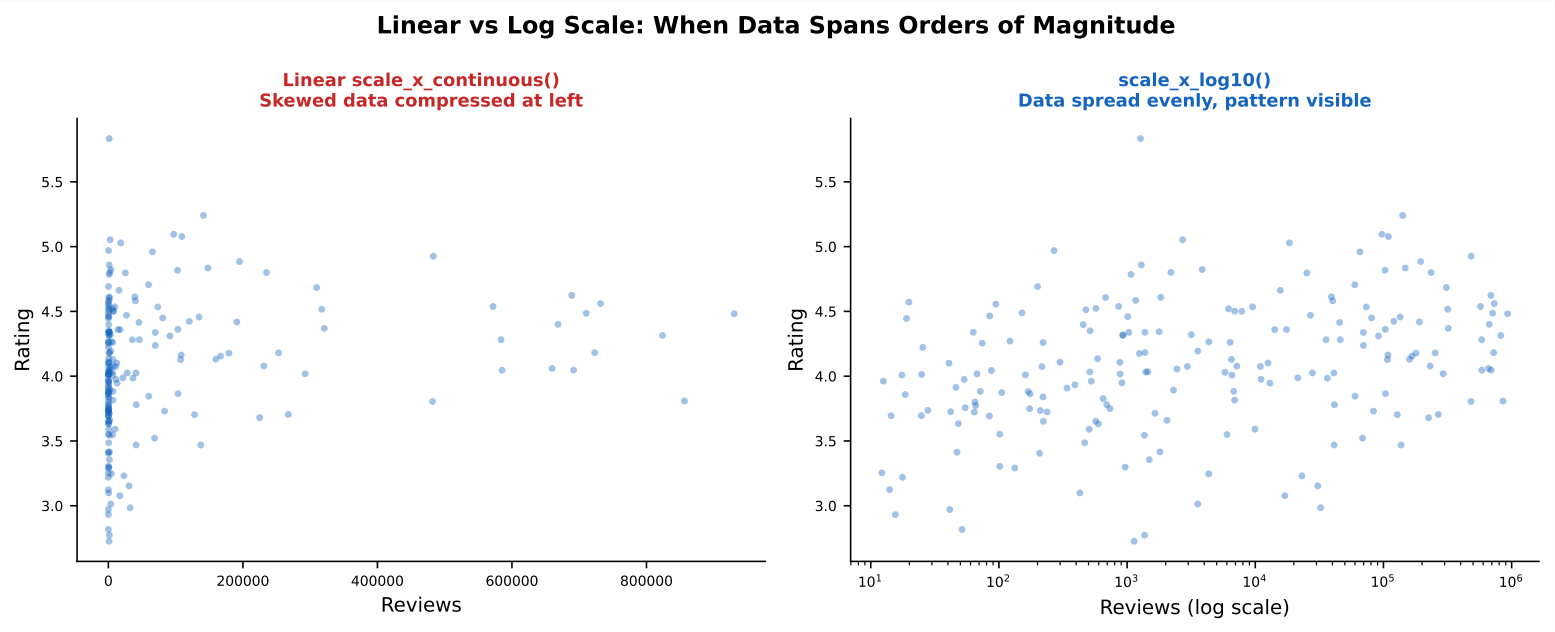
Scale Functions Cheat Sheet

Scale Functions Cheat Sheet

Scale Function	Purpose	Key Args
`scale_x_continuous()`	Numeric axis	`limits, breaks, labels, trans`
`scale_x_log10()`	Log-transformed axis	`Skewed distributions`
`scale_x_discrete()`	Categorical axis	`limits (reorder), labels`
`scale_x_date()`	Date axis	`date_breaks, date_labels`
`scale_color_manual()`	Custom discrete colors	`values = c(A='blue')`
`scale_color_brewer()`	ColorBrewer palette	`palette = 'Set2'`
`scale_color_viridis_c()`	Viridis continuous	`option = 'magma'`
`scale_color_gradient2()`	Diverging gradient	`midpoint, low, high`
`scale_fill_*()`	Fill (bars, areas)	`Same as color_* variants`
`scale_size()`	Point size	`range = c(1, 12)`

Position Scales

Linear vs Log Scale



Pro-Tip

Use `scale_x_log10()` (R) or `ax.set_xscale("log")` (Python) whenever data spans more than two orders of magnitude. The log transform converts multiplicative relationships to linear ones, making patterns visible.

Log Scale in R vs Python

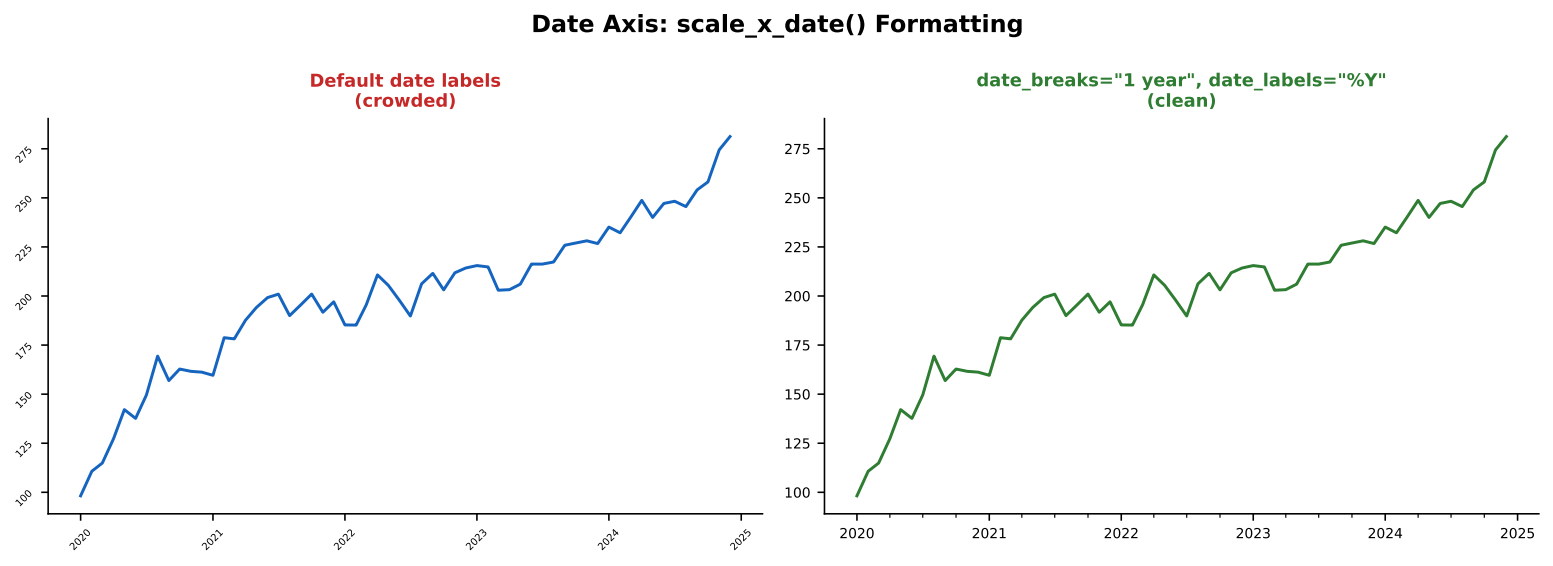
R (ggplot2)

```r
ggplot(apps, aes(x = Reviews,
                 y = Rating)) +
  geom_point(alpha = 0.3,
             size = 0.8) +

  # Log-transform x axis
  scale_x_log10(
    labels = scales::comma) +

  # Limit y without filtering
  coord_cartesian(
    ylim = c(1, 5)) +

  theme_minimal() +
  labs(x = "Reviews (log scale)")
```

Python (matplotlib)

```python
fig, ax = plt.subplots()

ax.scatter(reviews, rating,
    s=8, alpha=0.3)

# Log-transform x axis
ax.set_xscale("log")

# Zoom without filtering
ax.set_ylim(1, 5)

# Format tick labels
from matplotlib.ticker import (
    FuncFormatter)
ax.xaxis.set_major_formatter(
    FuncFormatter(lambda x, p:
        f"{x:,.0f}"))

ax.set_xlabel("Reviews (log)")
```

Date Axis Formatting



Date Scales in R vs Python

R

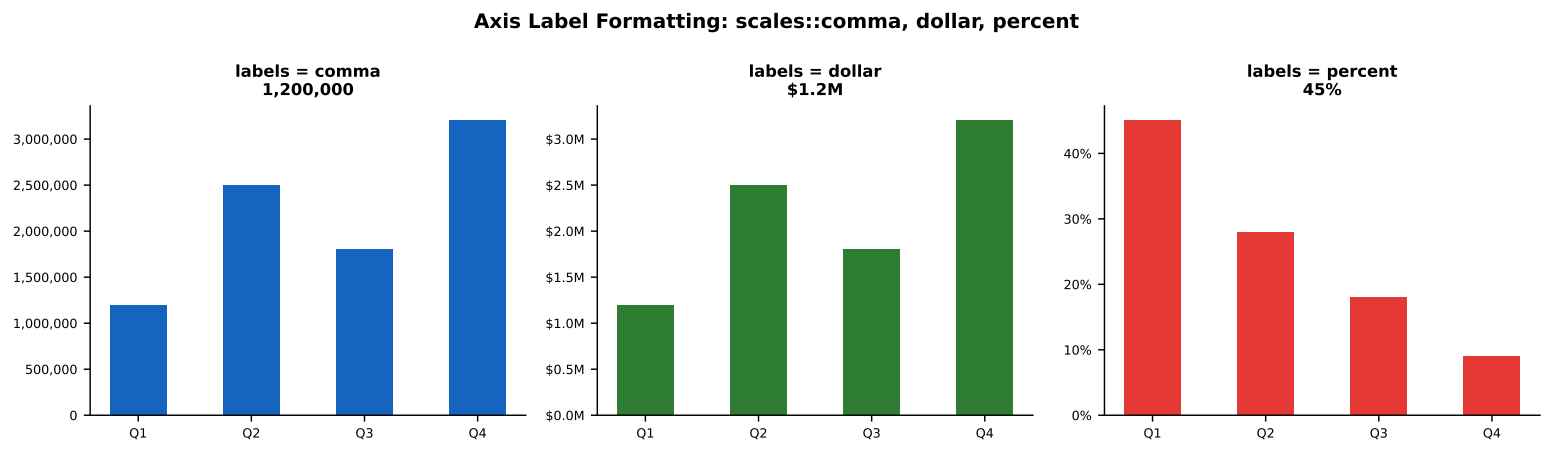
```r
ggplot(df, aes(x = date,
               y = value)) +
  geom_line() +
  scale_x_date(
    date_breaks = "6 months",
    date_labels = "%b %Y") +
  # Produces: "Jan 2023"
  theme_minimal()
```

Python

```python
import matplotlib.dates as mdates

ax.plot(dates, values)

ax.xaxis.set_major_locator(
    mdates.MonthLocator(
        bymonth=[1, 7]))
ax.xaxis.set_major_formatter(
    mdates.DateFormatter("%b %Y")
        )

fig.autofmt_xdate()
```

Colour Scales

Axis Label Formatting



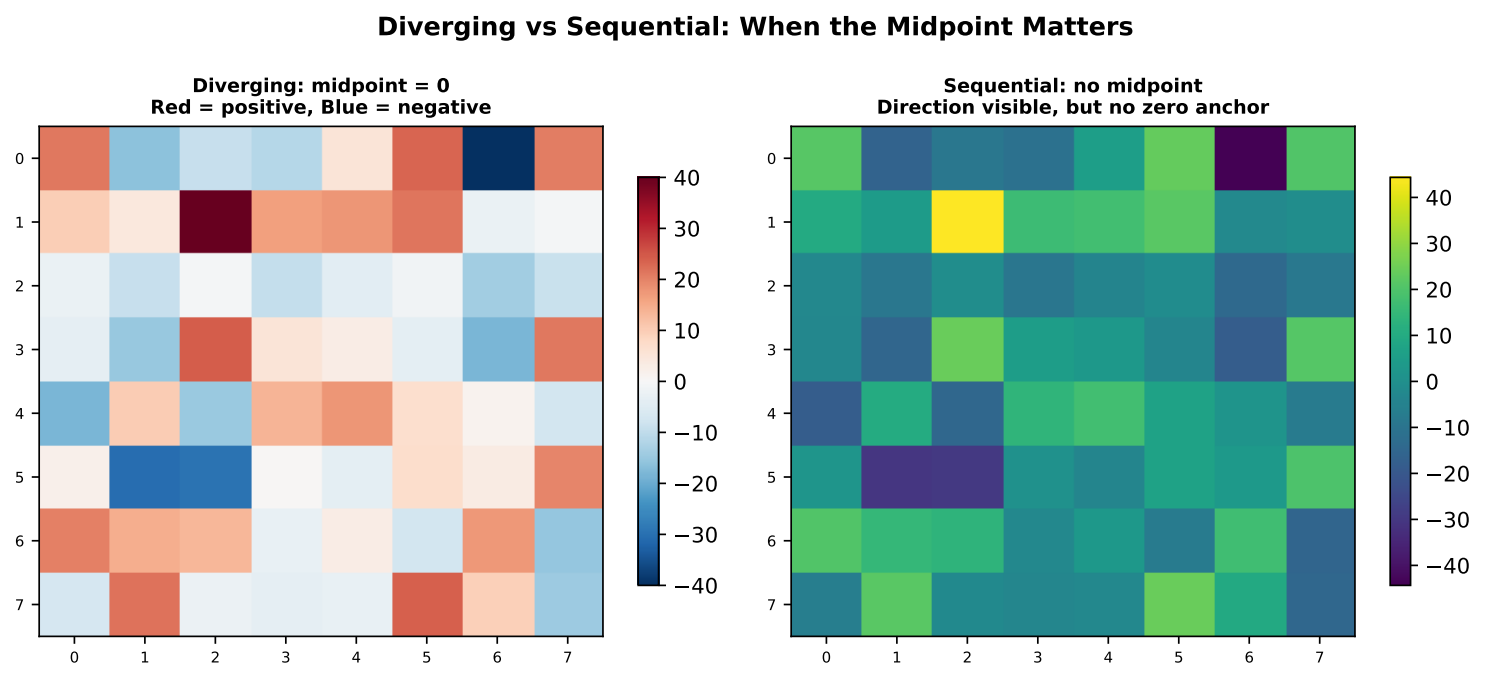
Pro-Tip
In R: `scale_y_continuous(labels = scales::comma)` or `scales::dollar` or `scales::percent`. In Python: `ax.yaxis.set_major_formatter(FuncFormatter(...))` or use `matplotlib.ticker.StrMethodFormatter`.

Colour Scale Selection

Data Type	R Scale	Python Equivalent
Categorical ($\leq$7)	`scale_color_brewer("Set2")`	`palette="Set2"`
Categorical (custom)	`scale_color_manual(values)`	loop + palette dict
Sequential cont.	`scale_color_viridis_c()`	`cmap="viridis"`
Diverging cont.	`scale_color_gradient2()`	`TwoSlopeNorm`
Binary	`scale_color_manual(c(a,b))`	2-colour list

Always match scale type to variable type. Using a sequential scale for categorical data implies a false ordering; using a qualitative scale for continuous data creates false boundaries.

Diverging vs Sequential

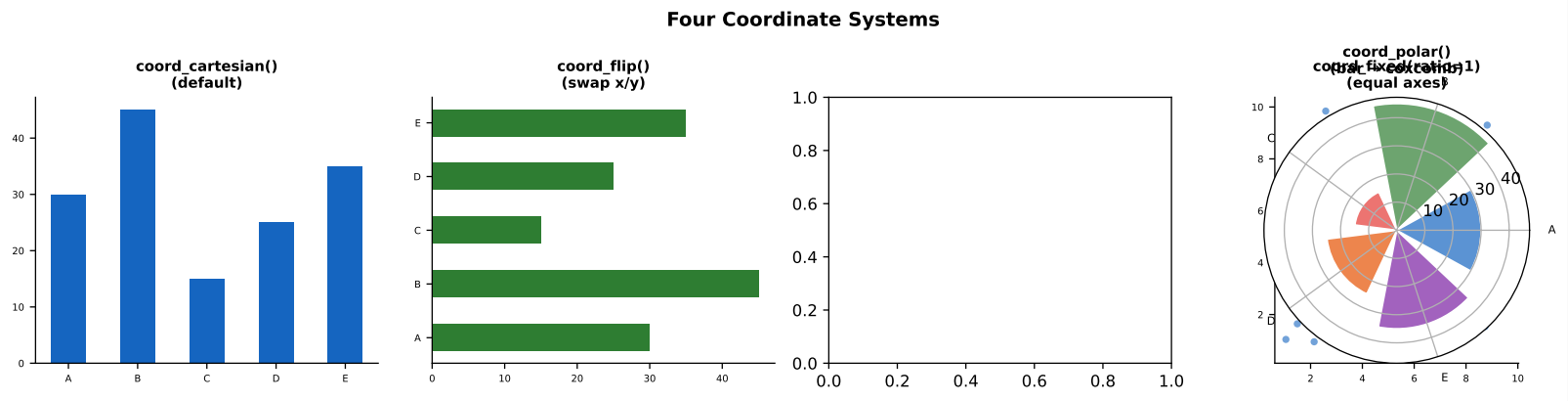


Data Insight

Use a diverging scale (`RdBu_r`, `gradient2`) when the data has a meaningful midpoint (zero, average, break-even). The two hues encode direction; the midpoint anchors neutral. Use sequential (`viridis`) when only magnitude matters.

Coordinate Systems

Four Coordinate Systems

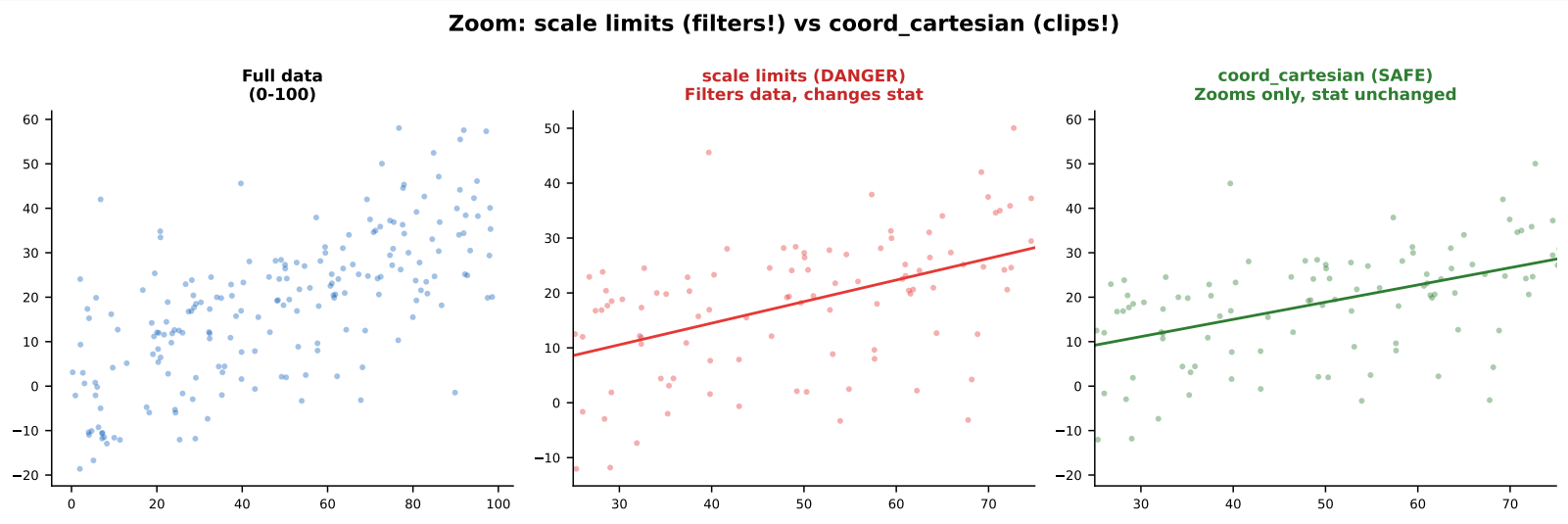


Coordinate Systems in Code

```
# Default: Cartesian
p + coord_cartesian()

# Flip x and y (horizontal bars)
p + coord_flip()

# Polar (bar -> coxcomb/pie)
p + coord_polar(theta = "y")    # pie: maps y to angle
p + coord_polar(theta = "x")    # coxcomb: maps x to angle

# Equal aspect ratio (maps, scatter comparison)
p + coord_fixed(ratio = 1)

# Python equivalents:
# coord_flip     -> ax.barh() instead of ax.bar()
# coord_polar    -> fig.add_subplot(projection="polar")
# coord_fixed    -> ax.set_aspect("equal")
```

Zooming Safely

scale limits vs coord_cartesian



The Zoom Rule

DANGEROUS: scale limits

```r
# Filters data outside range!
ggplot(df, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm") +
  scale_x_continuous(
    limits = c(25, 75))
# geom_smooth recalculated on
# subset -> different slope!
```

Data outside limits is **removed** before statistics are computed. The regression line changes.

SAFE: coord_cartesian

```r
# Zooms without filtering
ggplot(df, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm") +
  coord_cartesian(
    xlim = c(25, 75))
# geom_smooth computed on ALL
# data, then clipped to window
```

Data is only **clipped** at the viewport edge. Statistics remain unchanged.

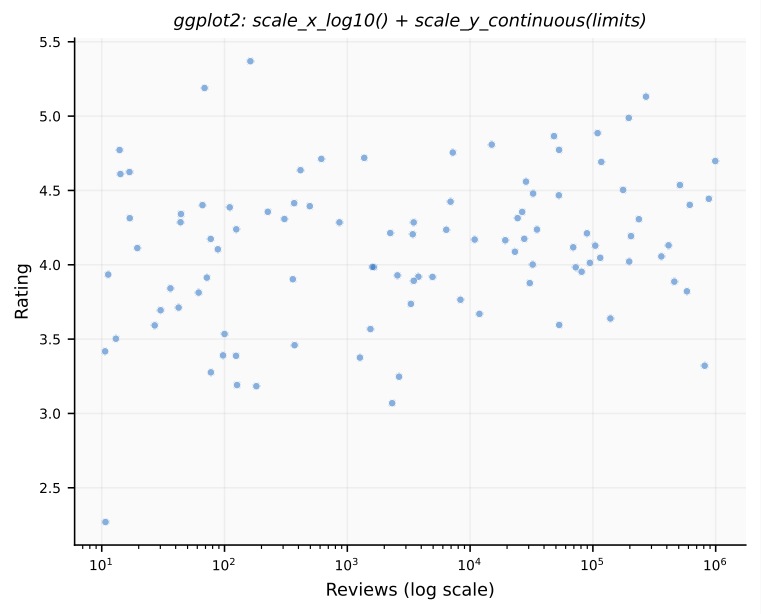
Pro-Tip

Always use `coord_cartesian()` for zooming. Use scale `limits` only when you intentionally want to **exclude** data from computation. In Python: `ax.set_xlim()` is equivalent to `coord_cartesian` — it clips, not filters.

Complete Examples

R: Combining Scale + Coord

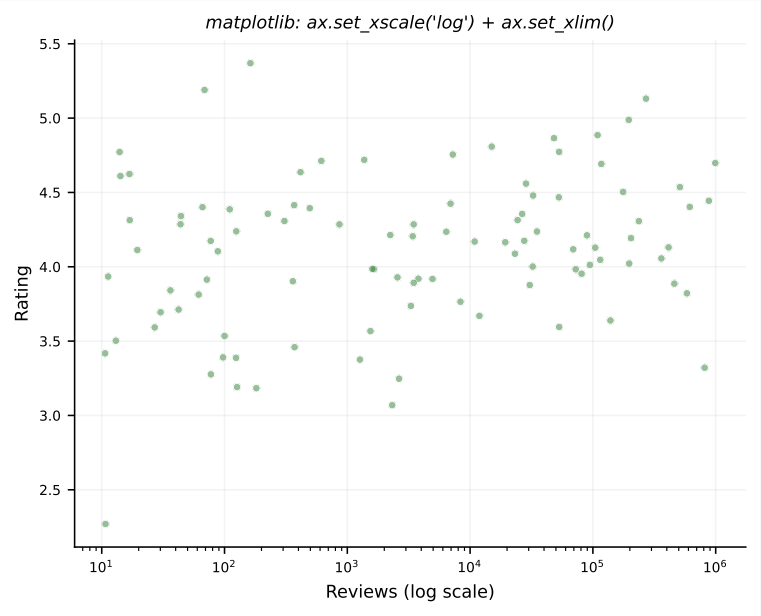
```r
ggplot(apps_clean,
  aes(x = Reviews, y = Rating,
      color = Type)) +
  geom_point(alpha = 0.3,
             size = 0.8) +

  # Position scale: log
  scale_x_log10(
    labels = scales::comma) +

  # Colour scale: manual
  scale_color_manual(
    values = c(Free = "#1565C0",
               Paid = "#E53935")) +

  # Zoom safely
  coord_cartesian(
    ylim = c(1, 5)) +

  theme_minimal() +
  labs(title = "Reviews vs Rating",
       x = "Reviews (log)",
       y = "Rating")
```



Three scale operations: (1) x-axis log-transformed, (2) colour manually assigned, (3) y-axis zoomed via coord.

Python: Same Three Operations

```python
fig, ax = plt.subplots(figsize
    =(6,5))

palette = {"Free": "#1565C0",
           "Paid": "#E53935"}
for t, c in palette.items():
    mask = df["Type"] == t
    ax.scatter(
        df.loc[mask, "Reviews"],
        df.loc[mask, "Rating"],
        s=8, c=c, alpha=0.3,
        edgecolors="none", label=t)

# Position scale: log
ax.set_xscale("log")

# Colour: handled by palette dict

# Zoom: set_ylim clips, not filters
ax.set_ylim(1, 5)

ax.legend(fontsize=7)
ax.set_xlabel("Reviews (log)")
ax.set_ylabel("Rating")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(
```



matplotlib equivalents: `set_xscale("log")` = log, palette dict = manual colour, `set_ylim()` = safe zoom.

Synthesis

Scale Selection Decision Tree

Question	Answer	Scale
Spans >2 orders?	Yes	`scale_x_log10()`
Has a meaningful zero?	Yes	`scale_color_gradient2()`
Categorical ≤7?	Yes	`scale_color_brewer()`
Dates on x?	Yes	`scale_x_date()`
Need horizontal bars?	Yes	`coord_flip()`
Need equal aspect?	Yes	`coord_fixed(ratio=1)`
Want to zoom?	Yes	`coord_cartesian(xlim)`

Key Takeaways

1. A **scale** maps data values (domain) through an optional transform to visual properties (range).
2. **Log scales** reveal patterns in skewed data; use when values span >2 orders of magnitude.
3. **Diverging** colour scales need a meaningful midpoint; **sequential** scales encode magnitude only.
4. Format axis labels with `scales::comma/dollar/percent` (R) or `FuncFormatter` (Python).
5. **coord_cartesian** zooms safely (clips); scale `limits` zoom dangerously (filters data).
6. In Python, `ax.set_xlim()` is a safe zoom (like `coord_cartesian`), not a filter.

Essential Readings

- Wickham, H. (2016). *ggplot2*, Chapters 6–8 (Scales, Colour, Coordinate Systems).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapters 3, 7, 8 (Coordinate Systems, Colour Scales).
- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapter 11 (Communication: Scales).

Next Module: Faceting & Small Multiples (W02-M05)